Animal_system.py

```
from abc import ABC,abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass
    def sleep(self):
        print("Animal is sleeping...")
class Dog(Animal):
    def sound(self):
        print("bark")
class Cat(Animal):
    def sound(self):
        print("meow")
class Cow(Animal):
    def sound(self):
        print("moo")
dog=Dog()
cat=Cat()
cow=Cow()

dog.sound()
dog.sleep()

cat.sound()
cat.sleep()

cow.sound()
cow.sleep()
```

Book.py

```
# Parent class
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def display_book_details(self):
        print("Title:", self.title)
        print("Author:", self.author)


# Child class
class IssuedBook(Book):
    def __init__(self, title, author, issued_to, issued_date):
        super().__init__(title, author)
        self.issued_to = issued_to
        self.issued_date = issued_date

    def display_issued_book_details(self):
        # Calling parent class method
        self.display_book_details()
        print("Issued To:", self.issued_to)
        print("Issued Date:", self.issued_date)


# Creating object of IssuedBook
issued_book = IssuedBook(
    "Python Basics",
    "John Smith",
    "Rahul",
    "05-02-2026"
)

# Display all details
issued_book.display_issued_book_details()
```

Str and repr_dunder_method.py

```
class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price


    def __str__(self):
        return f"Book: {self.title} by {self.author} - ■{self.price}"

    def __repr__(self):
        return f"Book('{self.title}', '{self.author}', {self.price})"



book1 = Book("Python Basics", "Hampana", 499)
book2 = Book("OOP Concepts", "John", 599)


print(book1)
print(book2)

book_list = [book1, book2]
print(book_list)
```

__eq__.py

```
class Mobile:
    def __init__(self, brand, model, price):
        self.brand = brand
        self.model = model
        self.price = price

    def __eq__(self, other):
        return self.brand == other.brand and self.model == other.model


m1 = Mobile("Samsung", "S23", 70000)
m2 = Mobile("Samsung", "S23", 75000)
m3 = Mobile("Apple", "iPhone 15", 80000)

print(m1 == m2)
print(m1 == m3)
```

bankaccountt.py

```
# Parent class
class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print("Deposited:", amount)

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print("Withdrawn:", amount)
        else:
            print("Insufficient balance")

    def display_balance(self):
        print("Account Holder:", self.account_holder)
        print("Balance:", self.balance)


# Child class 1
class SavingsAccount(BankAccount):
    def __init__(self, account_holder, balance, interest_rate):
        super().__init__(account_holder, balance)
        self.interest_rate = interest_rate

    def add_interest(self):
        interest = self.balance * self.interest_rate / 100
        self.balance += interest
        print("Interest added:", interest)


# Child class 2
class CurrentAccount(BankAccount):
    def __init__(self, account_holder, balance, overdraft_limit):
        super().__init__(account_holder, balance)
        self.overdraft_limit = overdraft_limit

    def withdraw_with_overdraft(self, amount):
        if amount <= self.balance + self.overdraft_limit:
            self.balance -= amount
            print("Withdrawn with overdraft:", amount)
        else:
            print("Overdraft limit exceeded")


# Creating objects
savings = SavingsAccount("Ravi", 5000, 5)
current = CurrentAccount("Anita", 3000, 2000)

# Testing SavingsAccount methods
print("\nSavings Account Details")
savings.display_balance()
savings.deposit(1000)
savings.add_interest()
savings.withdraw(2000)
savings.display_balance()

# Testing CurrentAccount methods
print("\nCurrent Account Details")
current.display_balance()
current.deposit(500)
```

```
current.withdraw_with_overdraft(4500)
current.display_balance()
```

call.py

```
class Calculator:

    def __call__(self, a, b):
        return a + b


obj = Calculator()

print(obj(10, 20))
```

college management.py

```
# Base class
class Person:
    def __init__(self, name):
        self.name = name

    def display_person(self):
        print("Name:", self.name)


# Student class (inherits from Person)
class Student(Person):
    def __init__(self, name, student_id):
        Person.__init__(self, name)
        self.student_id = student_id

    def display_student(self):
        self.display_person()
        print("Student ID:", self.student_id)


# SportsPlayer class (inherits from Person)
class SportsPlayer(Person):
    def __init__(self, name, sport_name):
        Person.__init__(self, name)
        self.sport_name = sport_name

    def display_sports_player(self):
        self.display_person()
        print("Sport Name:", self.sport_name)


# CollegeStudent class (inherits from Student and SportsPlayer)
class CollegeStudent(Student, SportsPlayer):
    def __init__(self, name, student_id, sport_name, college_name):
        Student.__init__(self, name, student_id)
        SportsPlayer.__init__(self, name, sport_name)
        self.college_name = college_name

    def display_college_student(self):
        self.display_person()
        print("Student ID:", self.student_id)
        print("Sport Name:", self.sport_name)
        print("College Name:", self.college_name)


# Creating object of CollegeStudent
cs = CollegeStudent("Rahul", "CS101", "Cricket", "ABC College")

# Displaying all details
cs.display_college_student()
```

contains.py

```
class Library:
    def __init__(self, books):
        self.books = books

    def __contains__(self, item):
        return item in self.books


library = Library(["Python", "Java", "C++"])

print("Python" in library)
print("HTML" in library)
```

decorator.py

```
def login(login_page):
    def wrapper(user,password):
        if user=="admin" and password=="1234":
            print("login successfull")
            login_page(user,password)
        else:
            print("login failed")
    return wrapper



@login
def login_page(user,password):
    print("welcome to the dasboard")
login_page("admin","1234")
```

decorator2.py

```
def admin_only(func):
    def wrapper(username):
        if username == "admin":
            return func(username)
        else:
            print("Access Denied")
    return wrapper


@admin_only
def dashboard(username):
    print("Welcome to Admin Dashboard!")



dashboard("admin")
dashboard("Hampana")
```

delete.py

```
class Session:
    def __del__(self):
        print("Session Ended")


obj = Session()

del obj
```

encapsulation.py

```
class InstagramAccount:
    def __init__(self, username, password):
        self.username = username
        self.__password = password
        self.__private_reels = []
        self.__archived_reels = []


    def add_private_reel(self, reel):
        self.__private_reels.append(reel)


    def add_archived_reel(self, reel):
        self.__archived_reels.append(reel)


    def show_private_reels(self, is_follower):
        if is_follower:
            print("Private Reels:", self.__private_reels)
        else:
            print("You are not a follower. Access denied.")


    def show_archived_reels(self, password):
        if password == self.__password:
            print("Archived Reels:", self.__archived_reels)
        else:
            print("Wrong password. Cannot access archived reels.")


    def set_password(self, old_password, new_password):
        if old_password == self.__password:
            self.__password = new_password
            print("Password updated successfully.")
        else:
            print("Old password is incorrect.")



account = InstagramAccount("john_doe", "1234")


account.add_private_reel("Gym Reel")
account.add_private_reel("Travel Reel")


account.add_archived_reel("Old Birthday Reel")
account.add_archived_reel("College Memories Reel")

print("\n--- Private Reels Access ---")
account.show_private_reels(is_follower=True)
account.show_private_reels(is_follower=False)

print("\n--- Archived Reels Access ---")
account.show_archived_reels("1234")
account.show_archived_reels("0000")

print("\n--- Update Password ---")
account.set_password("1234", "5678")

print("\n--- Archived Reels After Password Update ---")
account.show_archived_reels("5678")
account.show_archived_reels("1234")
```

exit.py

```
class DatabaseConnection:

    def __enter__(self):
        print("Database Connected")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Database Closed")


with DatabaseConnection():
    print("Performing Query...")
```

get and init.py

```
class ShoppingCart:
    def __init__(self, items):
        self.items = items
    def __getitem__(self, index):
        return self.items[index]
    def __setitem__(self, index, value):
        self.items[index] = value

cart = ShoppingCart(["Apple", "Banana", "Milk"])

print(cart[0])

cart[1] = "Orange"
print(cart.items)
```

getandset.py

```
class ShoppingCart:
    def __init__(self, items):
        self.items = items
    def __getitem__(self, index):
        return self.items[index]

    def __setitem__(self, index, value):
        self.items[index] = value


cart = ShoppingCart(["Shoes", "Bag", "Watch"])

print(cart[0])


cart[1] = "New Item"

print(cart.items)
```

gt and lt.py

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def __gt__(self, other):
        return self.salary > other.salary

    def __lt__(self, other):
        return self.salary < other.salary


e1 = Employee("John", 50000)
e2 = Employee("Alice", 60000)

print(e1 > e2)
print(e1 < e2)
```

instagram.py

```
class Instagram:
     def __init__(self,title, description,comments,creator_name,location):
         self.title = title
         self.description = description
         self.likes = 0
         self.comments=comments
         self.creator_name=creator_name
         self.location=location
     def display_title(self):
         print("The title of the reel is ",self.title)
     def display_description(self):
         print("The description of the reel is",self.description)
     def display_likes(self):
         print("The likes of the reel is ",self.likes)
     def liked(self):
         self.likes += 1
     def disliked(self):
         if self.likes > 0:
             self.likes-=1
     def display_comments(self):
         print("The comments of the reel is ",self.comments)
     def display_creator_name(self):
         print("The creator name of the reel is ",self.creator_name)
     def display_location(self):
         print("The location of the reel is ",self.location)
reel1=Instagram("dancing","dancing with friends","Nice Dance","Savita","Mysuru")
# 0
reel1.disliked() #0
reel1.liked() #1
reel2=Instagram("finance minister conference","finance minister conference with friends","Good","News First"

reel1.liked() #2
# 0
reel2.liked() #1
reel1.disliked() #1
reel1.display_likes()
reel2.display_likes()
print(id(reel1))
print(id(reel2))
reel1.display_comments()
reel2.display_comments()
reel1.display_creator_name()
reel2.display_creator_name()
reel1.display_location()
reel2.display_location()
```

iter and next.py

```
class Counter:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= self.end:
            num = self.current
            self.current += 1
            return num
        else:
            raise StopIteration


for i in Counter(1, 5):
    print(i)
```

mobile_dundder.py

```
class Mobile:
    def __init__(self, brand, model, price):
        self.brand = brand
        self.model = model
        self.price = price

    def __eq__(self, other):
        if isinstance(other, Mobile):
            return self.brand == other.brand and self.model == other.model
        return False


m1 = Mobile("Samsung", "S23", 70000)
m2 = Mobile("Samsung", "S23", 75000)
m3 = Mobile("Apple", "iPhone 15", 80000)

print(m1 == m2)
print(m1 == m3)
```

new_class.py

```
class User:

    def __new__(cls, *args, **kwargs):
        print("Object is being created")
        return super().__new__(cls)

    def __init__(self, name):
        self.name = name
        print("Object is initialized")


u1 = User("Hampana")
```

online shop(multiple in).py

```
# Parent Class
class Product:
    def __init__(self, product_name, price):
        self.product_name = product_name
        self.price = price

    def display_product(self):
        print("Product Name:", self.product_name)
        print("Price:", self.price)


# Child Class
class ElectronicProduct(Product):
    def __init__(self, product_name, price, brand, warranty):
        super().__init__(product_name, price)
        self.brand = brand
        self.warranty = warranty

    def display_electronic_product(self):
        print("Brand:", self.brand)
        print("Warranty:", self.warranty, "years")


# Grandchild Class
class MobilePhone(ElectronicProduct):
    def __init__(self, product_name, price, brand, warranty, ram, storage):
        super().__init__(product_name, price, brand, warranty)
        self.ram = ram
        self.storage = storage

    def display_mobile_details(self):
        print("RAM:", self.ram)
        print("Storage:", self.storage)


# Creating object of MobilePhone
mobile = MobilePhone(
    "Smartphone",
    25000,
    "Samsung",
    2,
    "8 GB",
    "128 GB"
)

# Displaying all details
print("---- Mobile Phone Details ----")
mobile.display_product()
mobile.display_electronic_product()
mobile.display_mobile_details()
```

payment.py

```
class Payment:
    def pay(self):
        print("Payment method selected")

class GooglePay(Payment):
    def pay(self):
        print("Payment done using Google Pay")

class PhonePe(Payment):
    def pay(self):
        print("Payment done using PhonePe")

class CreditCard(Payment):
    def pay(self):
        print("Payment done using Credit Card")


p = Payment()
gpay = GooglePay()
phonepe = PhonePe()
card = CreditCard()

p.pay()
gpay.pay()
phonepe.pay()
card.pay()
```

samrtphone.py

```
class Smartphone:
    def __init__(self, brand):
        self.brand = brand

    def make_call(self):
        print("Calling...")

class Camera(Smartphone):
    def take_photo(self):
        print(self.brand, "camera is taking a photo")

class MusicPlayer(Smartphone):
    def play_music(self):
        print(self.brand, "is playing music")


# Object Creation
phone1 = Camera("Samsung")
phone1.make_call()
phone1.take_photo()

phone2 = MusicPlayer("Apple")
phone2.make_call()
phone2.play_music()
```

student_admission.py

```
class Student:

    college_name = "MRIT College"

    # Constructor
    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no

    @classmethod
    def change_college(cls, new_name):
        cls.college_name = new_name

    @staticmethod
    def is_pass(marks):
        if marks >= 35:
            return "Pass"
        else:
            return "Fail"

    def display(self):
        print("Name:", self.name)
        print("Roll No:", self.roll_no)
        print("College:", Student.college_name)
        print("--------------------")


s1 = Student("Aishwarya", 101)
s2 = Student("Hampana", 102)

s1.display()
s2.display()

Student.change_college("MIT College")

print("After Changing College Name\n")

s1.display()
s2.display()

result1 = Student.is_pass(80)
result2 = Student.is_pass(90)

print("Marks 80:", result1)
print("Marks 90:", result2)
```

vehicle.py

```
from abc import ABC, abstractmethod
class Vehicle(ABC):

    @abstractmethod
    def start_engine(self):
        pass

class Car(Vehicle):
    def start_engine(self):
        print("Car engine started")
class Bike(Vehicle):
    def start_engine(self):
        print("Bike engine started")

class Bus(Vehicle):
    def start_engine(self):
        print("Bus engine started")


car = Car()
bike = Bike()
bus = Bus()

car.start_engine()
bike.start_engine()
bus.start_engine()
```